

Appendix: Measurement Methodology

Shared by report_nbody and report_pyflate — including the parts we got wrong first

Matan Cohen · Yuval Kogan · HWSW Final Project, Technion

This appendix holds the measurement machinery both benchmark reports depend on. It is separate because the two reports were repeating it, and because most of it is about **how we were wrong first** — which is the part worth writing down.

A1. What the guest PMU can and cannot do

The course VM is a QEMU/KVM guest, and for most of this project we believed its hardware performance counters were unusable. Our own scripts carried the comment “the `cycles` PMU event records zero samples — MUST use `-e cpu-clock`”, and both reports repeated it. **That was wrong**, and since it is a claim about hardware that a reader can check in one command, it needed correcting rather than quietly dropping.

Two separate effects had been conflated.

Counting works, including cycles

Three independent encodings of unhalted core cycles agree to within 1% on the same load:

event	count	note
<code>cycles</code>	180,523,907	fixed counter
<code>ref-cycles</code>	179,410,008	reference clock
<code>r003c</code>	178,709,534	<code>CPU_CLK_UNHALTED.THREAD_P</code> , general-purpose counter
<code>instructions</code>	538,815,265	for scale

The zeros that started the myth came from asking `perf stat` for **six** hardware events at once. The guest exposes **four** general-purpose counters (`generic registers: 4` in its `dmesg`), and rather than multiplexing, the vPMU returns 0 for the events that lose. Ask for four or fewer per pass and everything counts. Our scripts already split counters across two passes for this reason — the workaround was right, the explanation attached to it was not, and the committed counter data was never affected by the misunderstanding.

Sampling works too, but only with a fixed period

perf record invocation	samples
<code>-e cycles -F 999</code>	0
<code>-e cycles -F 4000</code>	0
<code>-e cycles -c 1000000</code>	2,315
<code>-e cycles -c 2000000</code>	9,735
<code>-e cpu-clock -F 999</code>	1,022

Frequency mode asks the kernel to **auto-tune** the sample period from observed counter feedback; that loop does not converge on the KVM vPMU, so the counter is never armed and the capture is silent. Pinning the period with `-c` sidesteps the tuning and the overflow-interrupt path works normally. The full production combination — `-e cycles -c 2000000 --call-graph dwarf,16384` against `python3-dbg` — yields 2,298 samples whose call chains resolve to real symbols end to end (`_start` → `Py_BytesMain` → `pymain_init` → ... → `__GI_setlocale`).

So this was a perf usage limitation, not a virtualization one, and no change to the VM is needed. We checked the alternative rather than assuming: the host is bare metal (Xeon E5-2630 v3, `systemd-detect-virt` reports `none`), `kvm.enable_pmu` is `Y`, and the guest already reports `Haswell events`, full-width counters, Intel PMU driver with the host’s full complement — so relaunching QEMU with `-cpu host,pmu=on` would change nothing. The one host-side knob left is `nmi_watchdog=1`, which pins one counter per CPU and is the standard thing to disable for guest PMU work; that needs root on the host, which we do not have.

Not everything is available: `LLC-load-misses` reports `<not supported>` in the guest, so last-level behaviour has to come from the generic `cache-references / cache-misses` pair.

Why the flame graphs still use `cpu-clock`. It is a software timer and samples in proportion to elapsed CPU time, which is the correct event for attributing **where** time goes. `cycles -c` is the right tool for the other half of the question, which is what A2 uses it for.

A2. A CPI stack, and the claim it overturned

Lecture 4 pairs the two tools deliberately: a flame graph shows **where** time goes but not why; a CPI stack shows **why** but not where; they are meant to be used together. Believing the PMU was unusable, we only ever had the first half.

The pyflate report asserted that `bwt_reverse` is “a 399 KB data-dependent pointer chase”, **irreducibly serial and memory-bound**, and built its processing-in-memory proposal on that. The claim was read off the source (`end = T[end]` in a loop) and never measured. With the PMU it is testable.

`dev/pyflate/phase_cpi.py` runs one pipeline stage per process so `perf stat` attributes cleanly. Note that `bwt_reverse` is really two different things — `bwt_transform()`, an $O(n)$ counting sort that is sequential and allocation-heavy, plus the chase — so the chase is also measured on its own with the transform hoisted out. Course VM, release `python3`, native decode back end, $n = 336,184$ chase steps per iteration:

phase	ms / iter	IPC	LLC miss rate	L1 miss rate	cycles / step
decode (native)	3.3	1.49	1.28%	2.20%	—
bwt (whole)	137.9	2.91	2.46%	1.13%	—
chase (isolated)	47.5	2.42	0.84%	2.41%	381
rle4	23.8	2.89	1.71%	—	—

The chase is not memory-bound. It sustains 2.42 instructions per cycle, and its last-level miss rate is the **lowest** of any phase — 0.035 LLC misses per chase step, i.e. it reaches DRAM roughly once every 29 steps. A latency-bound dependent-load chain looks like the opposite: IPC well under 1 and a miss on most steps. What the chase actually spends its 381 cycles per step on is **interpreter overhead** — about 922 instructions per `end = T[end]`, which is far more work than is needed to hide an L2/L3 hit.

Two consequences, and the second one matters for the hardware proposal:

1. **The structural claim survives.** The dependency `end = T[end]` genuinely is a serial chain and cannot be parallelized. That was never in doubt.
2. **The memory claim does not.** At this block size the working set fits comfortably in a 20 MB L3, so a processing-in-memory unit aimed at shortening DRAM round trips is solving a problem this workload does not have. What the measurement supports instead is narrower and better founded: the chase is interpreter-bound **today**, so the remaining software win is porting it to native code — and only **after** that does the serial dependency become the binding constraint, at L2/L3 latency rather than DRAM latency. The hardware that follows from this is a sequencer with tightly-coupled SRAM holding `T`, not a DRAM-side PIM. A PIM argument would only start to apply at block sizes whose `T` table leaves cache.

This is the clearest case in the project of a measurement contradicting an argument we found persuasive on paper, and it only became possible because the PMU turned out to work.

A3. Flame graph trimming

A flame graph is exactly as tall as the single deepest stack in the profile, however rare that stack is, and ours were pathological that way. Left alone the `perf` capture of stock `pyflate` ran to 176 rows and took more than a page on its own. Three reductions, applied by `tools/trim_folded.py` and driven by `tools/vm_remake_flames.sh`, bring them down. They differ in what they cost, and that distinction matters more than the sizes.

Dropping import-time samples (py-spy figures). `py-spy` starts sampling at process start, so it catches CPython importing `re`, `enum` and `collections` before the benchmark loop is entered. Those stacks are deep — `_find_and_load` → `_load_unlocked` → `exec_module` → `re._compile` — and there are only a handful, so they set the height while contributing nothing.

profile	samples	dropped as startup	max depth
pyspy_nbody_stock	304	5 (1.6%)	36 → 12
pyspy_nbody_opt	187	5 (2.7%)	41 → 16
pyspy_pyflate_stock	404	13 (3.2%)	61 → 18
pyspy_pyflate_opt	104	7 (6.7%)	60 → 15

This is not a truncation; it excludes a **phase** the figure was never meant to show.

Eliding the common prefix (perf figures). The C-level captures begin with interpreter start-up and the `pyperf` harness — 97 frames for `nbody`, 104 for `pyflate`, present in $\geq 95\%$ of samples. A frame in essentially every sample is constant context: no information, one row of height. Every surviving frame keeps its exact sample count, so **no width in the plot changes**; only the y-origin moves. The threshold is 95% rather than 100% because roughly 1.7% of DWARF unwinds are partial and start mid-interpreter, and at 100% those few fragments block the trim entirely — 178 rows became 168, i.e. nothing.

Capping depth (perf figures). Even without the prefix, a thin tower of deep stacks kept the figure near a full page, so stacks are capped at the shallowest depth leaving 95% of samples whole. *This one genuinely discards detail*: the tips of about 4.5% of samples are merged into their ancestors. It is the only one of the three that loses information, which is why each affected caption states the cut depth and the truncated share, and why the uncut graph is committed beside it as `*_full.svg`.

None of the three rescales anything, and **no number quoted in either report is derived from a trimmed graph** — timings come from `pyperf`, and self-time percentages from flat `perf` report output and cProfile, both on untrimmed data.

A4. Standing rules

- **python3-dbg timings are never quoted.** The debug build's assertions and allocator hooks inflate interpreter-internal frames roughly 2.5–3×. It is used only where symbols are needed, i.e. for profile **shape**.
- **perf record -g produces garbage here.** `python3-dbg` is built without frame pointers, so frame-pointer unwinding walks into freed stack memory and yields [unknown] frames and `0xfdfdfdfdfdfdf00` chains — the `Py_DEBUG` fill byte. `--call-graph dwarf,16384` fixes it, at ~100× larger captures. This was caught only by rasterizing a graph and looking at it; a broken flame graph still renders as perfectly valid SVG.
- **CPython 3.10 has no -X perf trampoline**, so `perf` can only ever show C frames. Python-level graphs come from `py-spy`.
- **pyperf system tune sets perf_event_max_sample_rate=1**, throttling `perf record` to 1 Hz. The profile stage resets it. Not persisted across guest reboots.
- **Pin the back end.** Both benchmarks use a Rust kernel when one is importable, so what gets measured otherwise depends on what happens to be installed. `HWSW_BACKEND` pins it and the choice is recorded in each result JSON's `hwsw_backend` metadata. `pyperf` re-executes workers with a scrubbed environment, so this needs `--inherit-environ HWSW_BACKEND` — without it both sides silently measure the same back end, which happened, and only the metadata caught it.
- **A stage stamp is not evidence.** A stale baseline once let a `compare` stage “succeed” against the previous session's data and produce a wrong number that looked entirely reasonable. Stages now validate that their output artifact exists, not merely that a stamp does.